



Amazon Athena

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Amazon Athena



Amazon Athena is an interactive query service that makes it easy to analyze data directly from S3



Athena is based off the open-source distributed query engine **Apache Presto**

Athena can do two things:

- **Athena SQL:** lets you run SQL queries on S3 Buckets
 - Athena uses **Tinro SQL** which is a fork of Apache Presto
 - Can commonly access via the AWS Management console to enter queries
 - JDBC or ODBC drivers to interact with Athena
 - Query with the AWS CLI or AWS SDKs
- **Apache Spark on Amazon Athena:** also interactively run data analytics using Apache Spark
 - Access via Jupyter compatible notebooks with Apache Spark



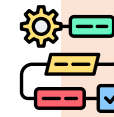
Athena is **serverless** so you only pay for what you use.

Athena integrates with the following AWS services:

- CloudFormation, CloudFront, CloudTrail, DataZone, ELB, EMR, AWS Glue Data Catalog, IAM, QuickSight, S3 Inventory, Step Functions, Systems Manager Inventory, VPC

Athena SQL Components

- **Workgroup** — saved queries which you grant permissions to other users to access
- **Data source** — a group of databases (sometimes called a catalog)
- **Database** — a group of tables (sometimes called a schema)
- **Table** — data organized as a group of rows or columns
- **Dataset** — the raw data of the table



The workflow for Athena is often to **dump the query results to the destination S3 Bucket.**

The screenshot shows the Amazon Athena console interface. The top navigation bar includes 'Editor', 'Recent queries', 'Saved queries', and 'Settings'. The 'Workgroup' is set to 'primary'. The left sidebar shows the 'Data' section with 'Data source' set to 'AwsDataCatalog' and 'Database' set to 'default'. The 'Tables and views' section shows a table named 'cloudtrail_logs_my_cloudtrail_ab_1212' with columns 'eventversion' and 'useridentity'. The main query editor shows a SQL query: 'select * from AwsDataCatalog.default.cloudtrail_logs_my_cloudtrail_ab_1212 limit 10'. The bottom of the console has buttons for 'Run', 'Explain', 'Cancel', 'Clear', and 'Create', along with a 'Reuse query results' toggle.

Data Definition Language (DDL)

A subset of SQL to define schema.
eg CREATE, ALTER, DROP

Data Manipulation Language (DML)

A subset of SQL to manipulate datasets.
eg. INSERT, UPDATE, DELETE

Data Query Language (DQL)

A subset of SQL to select datasets.
Ee. SELECT

Athena SQL Data Types

- **Boolean** — true or false
- **tinyint** — 8-bit signed integer, -2^7 (-128) to 2^7-1 (127)
- **Smallint** — 16-bit signed integer -2^{15} (-32,768) to $2^{15}-1$ (32,767)
- **Integer** — 32-bit signed integer -2^{31} to $2^{31}-1$ (~2.14 B in either direction)
 - Int — Used in DDL queries for Create Table
 - Integer Used in DQL queries for SELECT
- **Bigint** — -2^{63} to $2^{63}-1$ (9.2 quintillion in either direction)
- **Float** — 32-bit signed single-precision floating point number
- **Double** — 64-bit signed double-precision floating point number
- **Decimal(precision, scale)** — you specify the extra precision and scale, both values max to 38.
- **Char(n)** — fixed length character data expected provided by n. eg. char(3) = cat, dog, car
- **Varchar(n)** — Variable length character data between 1 and 65535
- **String** — A string literal enclosed in single or double quotes (String HIVE data type)
- **Ippaddress** — Represents an IP address in DML queries. Not supported for DDL
- **Binary** — used in parquet
- **Date** — in ISO format, such as YYYY-MM-DD
- **Timestamp** — Date and time instant in a java.sql.Timestamp eg. '2008-09-15 03:04:05.324'
- **Array<data type>** — An array of a data type eg. ARRAY[1,2,3]
- **Map<primitive_type, data_type>** — a map of values eg. MAP(ARRAY['foo', 'bar'], ARRAY[1, 2])
- **Struct<col_name : data_type, ...>** — A collection of elements of different component types.

Athena SQL – Table

Tables can be created two ways:

- Using SQL create table statement
- Using AWS Glue Wizard

Tables can be created automatically using AWS Glue crawler will crawl the data to produce a table schema.

Athena tables are AWS Glue Data Catalog tables and so they will exist in both services when creating an Athena table.

When you query FROM you'll use **AWSDataCatalog**

```
SELECT *
FROM "AWSDataCatalog"."your_database_name"."your_table_name"
WHERE "your_column_name" = 'your_value'
LIMIT 10;
```

Tables are likely to be created in the default database called “**default**”

Using SQL you'll specify:

- How to parse each row of data (possibly with regex)
- Specific the location of the data.

```
CREATE EXTERNAL TABLE IF NOT EXISTS cloudfront_logs (  
    `Date` Date,  
    Time STRING,  
    Location STRING,  
    Bytes INT,  
    RequestIP STRING,  
    Method STRING,  
    Host STRING,  
    Uri STRING,  
    Status INT,  
    Referrer STRING,  
    OS String,  
    Browser String,  
    BrowserVersion String  
) ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.RegexSerDe'  
WITH SERDEPROPERTIES (  
    "input.regex" = "^([?!#])([^\ ]+)\s+([^\ ]+)\s+([^\ ]+)\s+([^\ ]+)\s+  
    ([^\ ]+)\s+([^\ ]+)\s+([^\ ]+)\s+([^\ ]+)\s+([^\ ]+)\s+([^\ ]+)\s+  
    [^\\(\\)[\\(\\(\\^\\;]+).*(%20|[^\ ]+)[\\/](.*)$" )  
) LOCATION 's3://athena-examples-MyRegion/cloudfront/plaintext/';
```

Athena SQL – SerDe

SerDe is a serialization and deserialization libraries for parsing data from different data formats, such as CSV, JSON, Parquet, and ORC

It is the SerDe you specify, and not the DDL, that defines the table schema.

In other words, the SerDe can override the DDL configuration that you specify in Athena when you create your table.

There are several built-in SerDe supported by Athena.

```
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe' # csv
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde' # csv
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.avro.AvroSerDe' # avro
ROW FORMAT SERDE 'com.amazonaws.glue.serde.GrokSerDe' #grok
ROW FORMAT SERDE 'org.apache.hive.hcatalog.data.JsonSerDe' #json
ROW FORMAT SERDE 'org.openx.data.jsonserde.JsonSerDe' # json
ROW FORMAT SERDE 'com.amazon.ionhiveserde.IonHiveSerDe' #json
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.RegexSerDe' # regex
STORED AS ORC # ORC
STORED AS PARQUET # Parquet
```

Each SerDe is **configured differently**

```
ROW FORMAT SERDE 'org.apache.hadoop.hive.  
WITH SERDEPROPERTIES (  
    "input.regex" = "^(?!#)([^\n]+\n+([^\n]  
    ([^\n]+\n+([^\n]+\n+([^\n]+\n+([^\n]+  
    ]+)\n+([^\n]+\n+[^\n\]( [^\n;]+).*%\n    (.*)$"  
)
```